Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV

Assignment No 8

Name: Soham Kulkarni
PRN: 124B1F056
Subject: Design and Analysis of Algorithms
Batch: A3
GitHub Link: https://github.com/SOHAM0007-CODER/DAA_ASSIGNMNET

1. Topic

Find a SwiftShip delivery tour that starts at a chosen city, visits every other city once, and returns to the start. Minimize fuel cost within a total travel-time deadline using Least-Cost Branch and Bound instead of checking every city order without pruning.

2. Course Objective

- Understand the computational complexity of algorithms.
- Select a suitable algorithm design strategy for a practical problem.

3. Course Outcome

- Apply admissible bounds and a min-priority queue to exact TSP search.
- Recover the optimal feasible tour and analyze pruning and worst-case cost.

Theory

The Traveling Salesperson Problem seeks a tour through all cities with one final return to the start. Each road has a distance, fuel-consumption rate and travel time. The program minimizes total fuel expense, while the complete tour, including its return edge, must meet the time limit.

Least-Cost Branch and Bound stores partial routes in a min-priority queue and expands the one with the smallest cost lower bound. A feasible complete route gives an upper bound. Branches that cannot improve it or meet the deadline are discarded. Exact integer milli-INR costs avoid floating-point pruning errors.

Working of the Algorithm

- 1. Print directed road distances, fuel use, costs and times.
- 2. Try greedy seed tours and report root cost/time lower bounds.
- 3. Expand least-bound partial routes; trace the first 20 expanded nodes.
- 4. Print the proven feasible optimum, per-leg totals, deadline check and search statistics.

Complexity Analysis

Bounding takes O(N squared) per generated partial route. For G generated prefixes and peak queue size Q, search takes O(G * (N squared + log(Q+1))). Pruning may greatly reduce G, but its worst-case growth remains factorial.

Time Complexity

Best Case: O(N squared) when a feasible seed matches the root lower bound.

Average Case: Depends on the graph, deadline and bound strength; no fixed polynomial guarantee.

Worst Case: Factorial search, with bounding and priority-queue overhead.

Space Complexity

O(N squared + Q*N) for road matrices and queued partial paths. Hard 20+ city inputs may still require substantial time and memory.

Algorithm - Steps

- Step 1: Read roads, fuel price, starting city and total time limit.
- Step 2: Use a min-priority queue to expand the least-bound partial route.
- Step 3: Prune impossible or uncompetitive routes; print the proven optimum or infeasibility.

Pseudocode

```
initialize bestCost from any feasible greedy seed
insert the root if its bounds allow improvement
while the queue is not empty:
    remove the node with the least cost lower bound
    if its bound >= bestCost: stop
    extend to each unvisited reachable city
    test complete tours including the return edge
    queue only children that can improve and meet the deadline
```

Flow of Execution

START -> Read roads -> Seed upper bound -> Expand least-bound node -> Prune or branch -> Prove best tour -> END.

6. Example

At fuel price 100 INR/litre and a 90-minute limit, route 1 -> 3 -> 2 -> 4 -> 1 uses 5.5 litres, costs 550 INR and takes 75 minutes. A 130-minute limit permits the cheaper 1 -> 2 -> 3 -> 4 -> 1 tour: 400 INR and 120 minutes. A 70-minute limit has no feasible tour.

Conclusion

LC Branch and Bound proves an optimal fuel-cost tour under the entered time constraint when the search finishes. Safe lower bounds reduce unnecessary work, while worst-case factorial complexity remains. Road data are a static snapshot; changed conditions require a fresh run.

Assignment 8 | SAMPLE INPUT (input.txt)

```
4
100
12
1 2 10 100 30
1 3 15 100 15
1 4 25 200 20
2 1 20 200 20
2 3 10 100 30
2 4 15 100 15
3 1 20 200 20
3 2 15 100 15
3 4 10 100 30
4 1 10 100 30
4 2 20 200 20
4 3 20 200 20
1 90
```

Assignment 8 | FULL PROGRAM OUTPUT

```
ASSIGNMENT 8: SWIFTSHIP - LC BRANCH AND BOUND TSP
Enter number of cities (2-25):
Enter fuel price (whole INR per litre):
Enter number of directed roads:
Enter: from to distanceKm fuelMlPerKm travelMinutes
Enter start city and maximum total travel minutes:

DIRECTED ROAD TABLE
From -> To | Distance km | Consumption mL/km | Fuel L | Cost INR | Time min
1 -> 2 | 10 | 100 | 1.000 | 100.000 | 30
1 -> 3 | 15 | 100 | 1.500 | 150.000 | 15
1 -> 4 | 25 | 200 | 5.000 | 500.000 | 20
2 -> 1 | 20 | 200 | 4.000 | 400.000 | 20
2 -> 3 | 10 | 100 | 1.000 | 100.000 | 30
2 -> 4 | 15 | 100 | 1.500 | 150.000 | 15
3 -> 1 | 20 | 200 | 4.000 | 400.000 | 20
3 -> 2 | 15 | 100 | 1.500 | 150.000 | 15
3 -> 4 | 10 | 100 | 1.000 | 100.000 | 30
4 -> 1 | 10 | 100 | 1.000 | 100.000 | 30
4 -> 2 | 20 | 200 | 4.000 | 400.000 | 20
4 -> 3 | 20 | 200 | 4.000 | 400.000 | 20
Objective: minimum fuel cost | Starting city: 1 | Deadline: 90 minutes

INITIAL BOUNDS
Root cost lower bound:
400.000 INR
Root travel-time lower bound:
65 minutes
Initial feasible seed cost:
550.000 INR
Greedy seeds give upper bounds only; LC search proves the result.

SEARCH TRACE (first 20 expanded nodes)
Node | Partial route | Paid INR | Elapsed min | Lower bound INR
1 | 1 | 0.000 | 0 | 400.000
2 | 1 2 | 100.000 | 30 | 400.000
3 | 1 3 | 150.000 | 15 | 500.000

Optimal route:
1 -> 3 -> 2 -> 4 -> 1
LEG DETAILS
Leg | From -> To | Distance km | Fuel L | Cost INR | Time min | Cumulative INR
1 | 1 -> 3 | 15 | 1.500 | 150.000 | 15 | 150.000
2 | 3 -> 2 | 15 | 1.500 | 150.000 | 15 | 300.000
```

Assignment 8 | FULL PROGRAM OUTPUT (continued)

```
3 | 2 -> 4 | 15 | 1.500 | 150.000 | 15 | 450.000
4 | 4 -> 1 | 10 | 1.000 | 100.000 | 30 | 550.000
Cities visited exactly once: 4 / 4 | Return to start: YES
Deadline check: PASS | Spare time: 15 minutes

Minimum fuel cost: 550.000 INR
Total distance: 55 km
Total fuel: 5.500 litres
Total travel time: 75 minutes
Time limit: 90 minutes
Optimality proven by LC Branch and Bound.
Expanded nodes: 3
Pruned child nodes: 5
Peak queued nodes: 2
```